

HEALTHCARE • SALESFORCE PLATFORM

Appointment Booking & Patient Management System

A connected scheduling platform for hospitals and clinics — booking, reminders, doctor schedules, and real-time EMR sync.

Executive Project Overview

Objective

A unified system so patients, doctors, and admins operate from one source of truth.



Patients Book Appointments

Self-service scheduling with real-time slot availability.



Automated Reminders

Patients receive timely reminders ahead of visits.



Doctor Schedule Visibility

Doctors view their upcoming appointments at a glance.



Admin Slot Management

Admins manage, block, and reschedule time slots centrally.

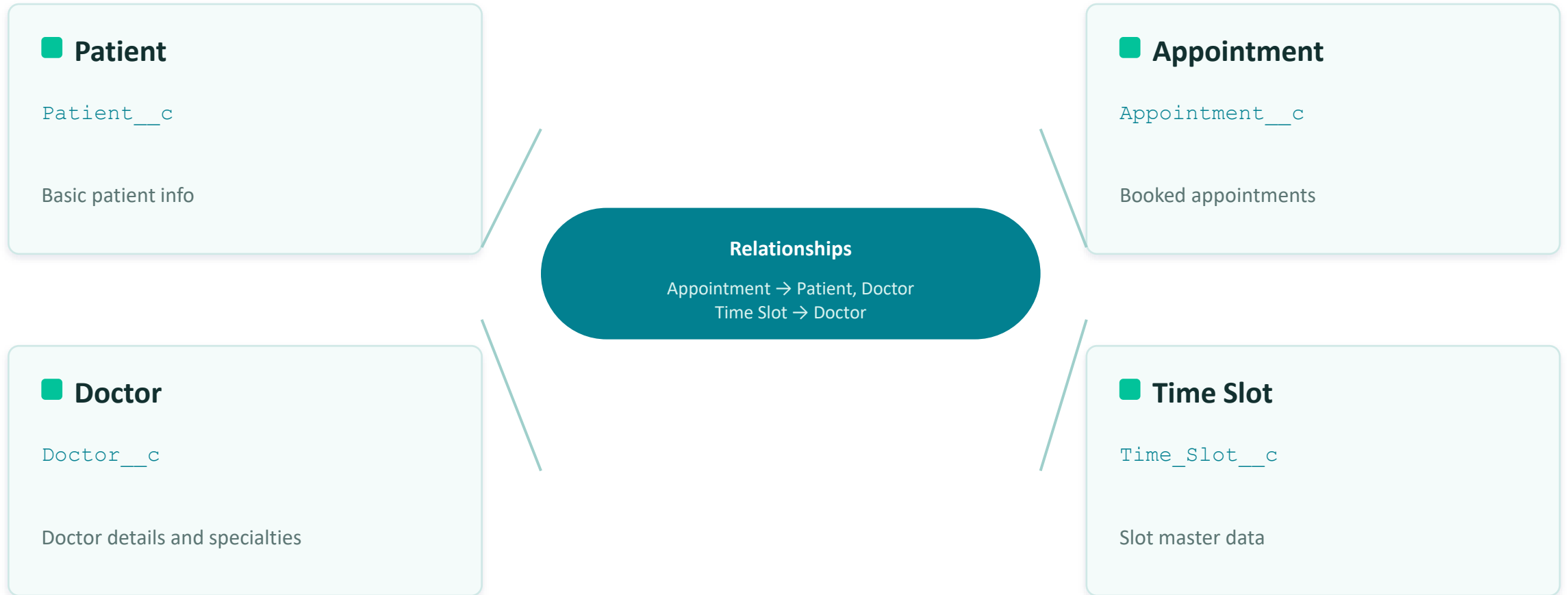


EMR Synchronization

Appointment data pushed automatically to the external EMR.

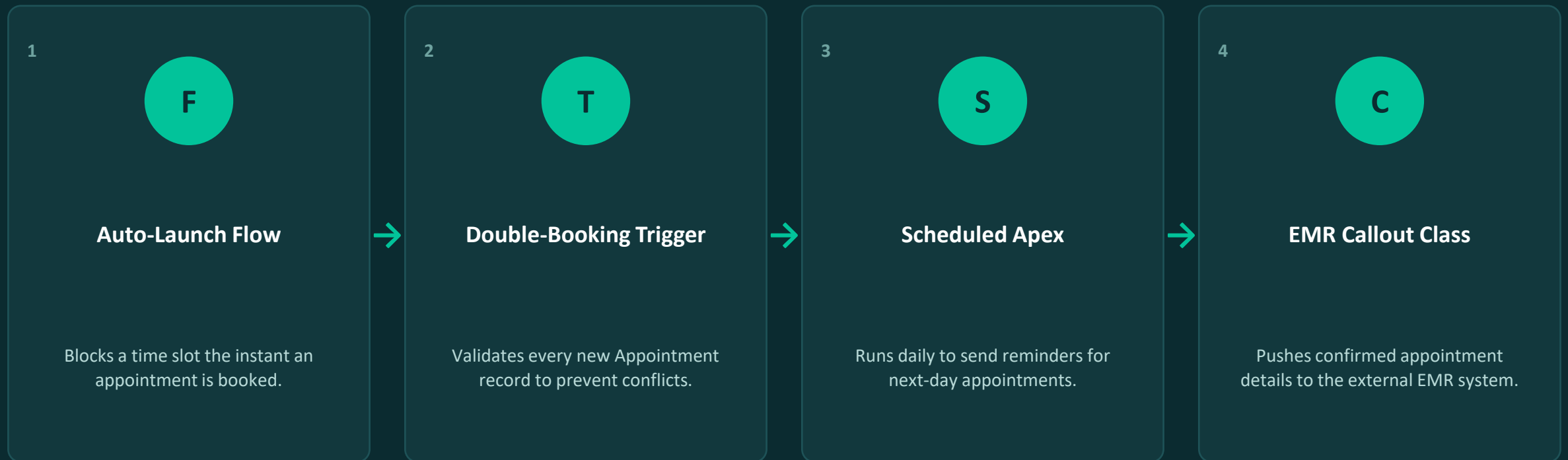
Data Model

Four core objects capture patients, doctors, slots, and bookings.



Automation & Apex Logic

Backend logic keeps bookings accurate, timely, and synced externally.



Flow — Auto-Block Time Slot

Record-triggered flow that runs the moment an appointment is booked.

1 Trigger

Fires automatically after an Appointment__c record is saved.

2 Condition

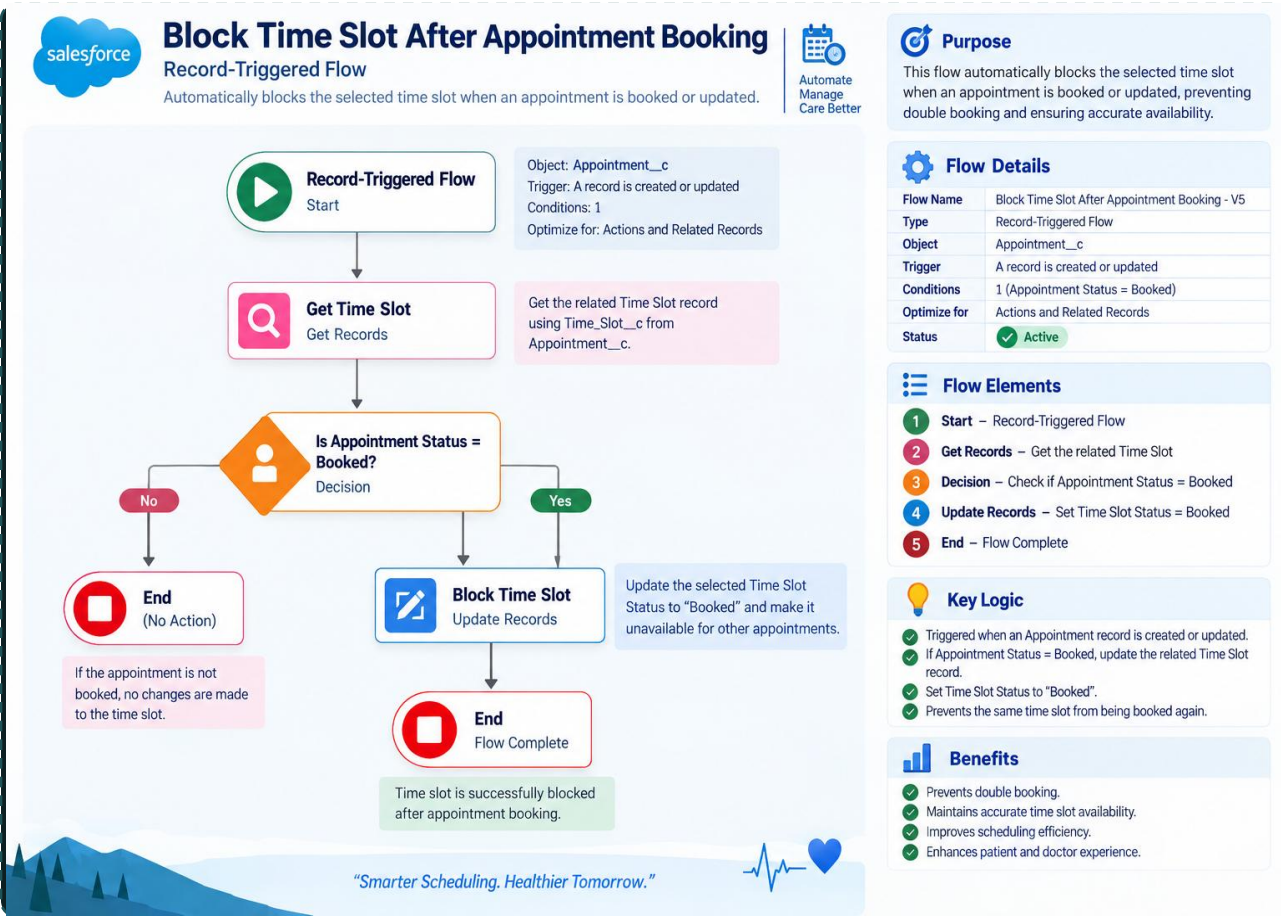
Only runs when the appointment Status is set to "Booked".

3 Action

Marks the related Time Slot as unavailable so it can't be double-booked.

4 Outcome

Patients always see a live, accurate list of open slots.



Trigger — Double-Booking Check

An Apex trigger on Appointment__c that guards against scheduling conflicts.

1 Trigger Event

Runs before insert and before update on Appointment__c.

2 Check

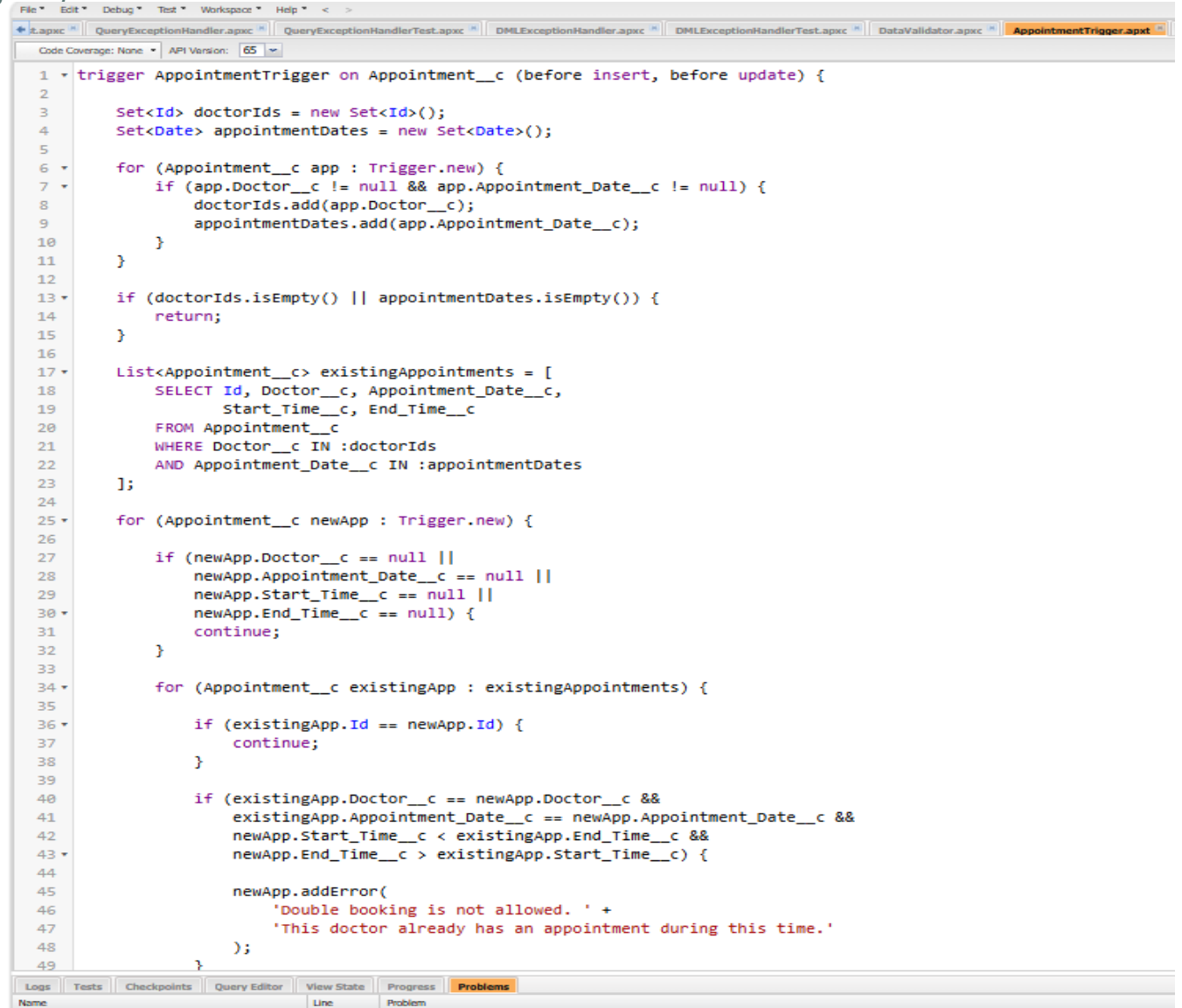
Looks for an existing appointment with the same Doctor and Time Slot.

3 Validation

Blocks the save and shows an error if a conflict is found.

4 Why It Matters

Guarantees no two patients can ever book the same slot.



```
1 trigger AppointmentTrigger on Appointment__c (before insert, before update) {
2
3     Set<Id> doctorIds = new Set<Id>();
4     Set<Date> appointmentDates = new Set<Date>();
5
6     for (Appointment__c app : Trigger.new) {
7         if (app.Doctor__c != null && app.Appointment_Date__c != null) {
8             doctorIds.add(app.Doctor__c);
9             appointmentDates.add(app.Appointment_Date__c);
10        }
11    }
12
13    if (doctorIds.isEmpty() || appointmentDates.isEmpty()) {
14        return;
15    }
16
17    List<Appointment__c> existingAppointments = [
18        SELECT Id, Doctor__c, Appointment_Date__c,
19            Start_Time__c, End_Time__c
20        FROM Appointment__c
21        WHERE Doctor__c IN :doctorIds
22        AND Appointment_Date__c IN :appointmentDates
23    ];
24
25    for (Appointment__c newApp : Trigger.new) {
26
27        if (newApp.Doctor__c == null ||
28            newApp.Appointment_Date__c == null ||
29            newApp.Start_Time__c == null ||
30            newApp.End_Time__c == null) {
31            continue;
32        }
33
34        for (Appointment__c existingApp : existingAppointments) {
35
36            if (existingApp.Id == newApp.Id) {
37                continue;
38            }
39
40            if (existingApp.Doctor__c == newApp.Doctor__c &&
41                existingApp.Appointment_Date__c == newApp.Appointment_Date__c &&
42                newApp.Start_Time__c < existingApp.End_Time__c &&
43                newApp.End_Time__c > existingApp.Start_Time__c) {
44
45                newApp.addError(
46                    'Double booking is not allowed. ' +
47                    'This doctor already has an appointment during this time.'
48                );
49            }
50        }
51    }
52 }
```

Scheduled Apex — Daily Reminders

A batch job that runs once a day to notify patients ahead of their visit.

1 Schedule

Runs automatically on a daily job, typically once every evening.

2 Query

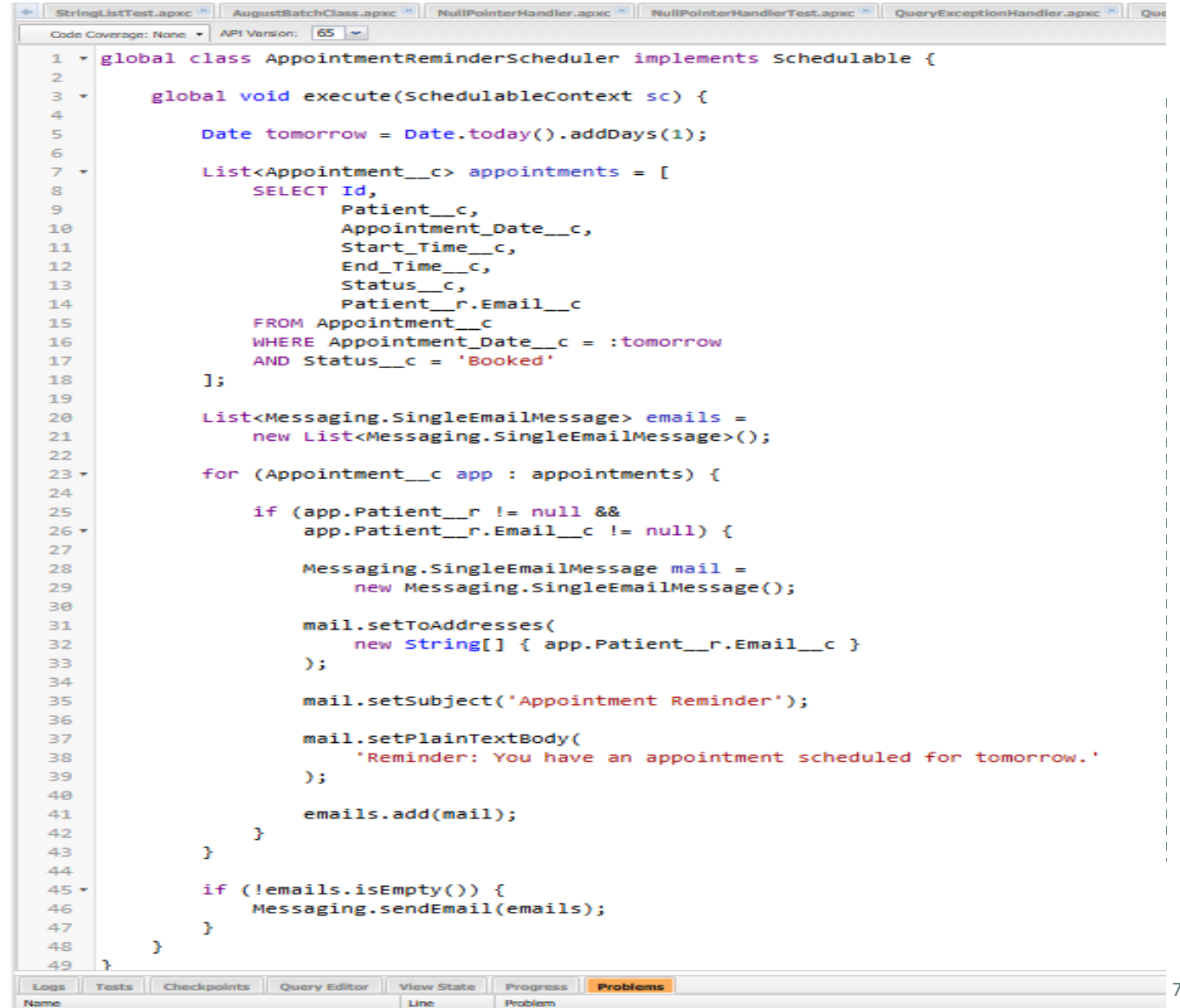
Finds every appointment scheduled for the next day.

3 Action

Sends a reminder (email or SMS) to each patient found.

4 Benefit

Reduces no-shows without any manual follow-up.



```
StringListTest.apxc | AugustBatchClass.apxc | NullPointerHandler.apxc | NullPointerHandlerTest.apxc | QueryExceptionHandler.apxc | Que
Code Coverage: None | API Version: 65
1 global class AppointmentRemindersScheduler implements Schedulable {
2
3     global void execute(SchedulableContext sc) {
4
5         Date tomorrow = Date.today().addDays(1);
6
7         List<Appointment__c> appointments = [
8             SELECT Id,
9                 Patient__c,
10                Appointment_Date__c,
11                Start_Time__c,
12                End_Time__c,
13                Status__c,
14                Patient__r.Email__c
15            FROM Appointment__c
16            WHERE Appointment_Date__c = :tomorrow
17            AND Status__c = 'Booked'
18        ];
19
20        List<Messaging.SingleEmailMessage> emails =
21            new List<Messaging.SingleEmailMessage>();
22
23        for (Appointment__c app : appointments) {
24
25            if (app.Patient__r != null &&
26                app.Patient__r.Email__c != null) {
27
28                Messaging.SingleEmailMessage mail =
29                    new Messaging.SingleEmailMessage();
30
31                mail.setToAddresses(
32                    new String[] { app.Patient__r.Email__c }
33                );
34
35                mail.setSubject('Appointment Reminder');
36
37                mail.setPlainTextBody(
38                    'Reminder: You have an appointment scheduled for tomorrow.'
39                );
40
41                emails.add(mail);
42            }
43        }
44
45        if (!emails.isEmpty()) {
46            Messaging.sendEmail(emails);
47        }
48    }
49 }
```

Callout Class — EMR Integration

Pushes confirmed appointment data out to the hospital's external EMR system.

1 Trigger Point

Fires once an appointment is confirmed/booked.

2 Callout

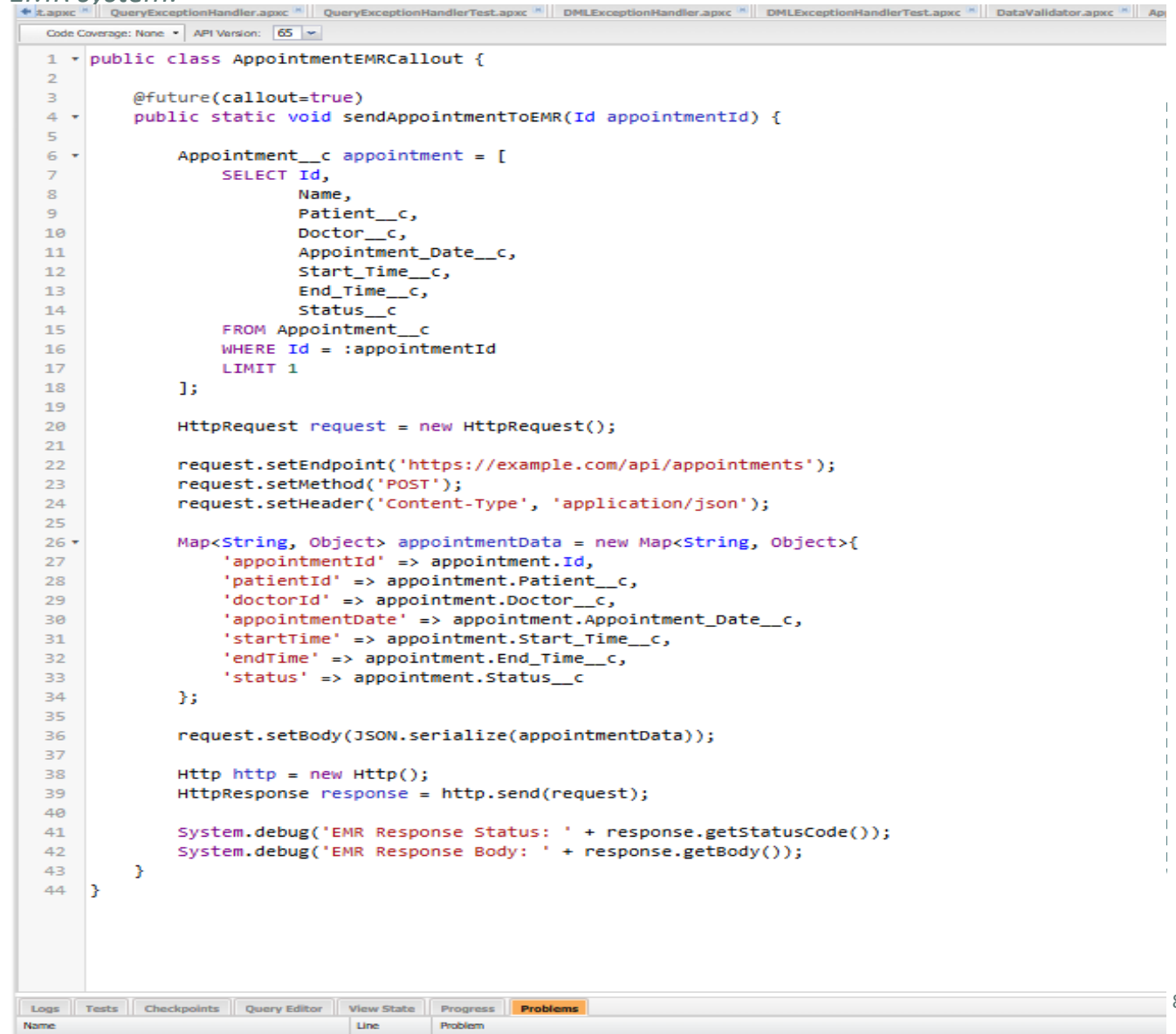
Sends an HTTP request with appointment details to the EMR's API.

3 Async Execution

Runs as a future or queueable call so it never slows down the booking screen.

4 Outcome

Clinical systems and the scheduling system always stay in sync.



```
1 public class AppointmentEMRCallout {
2
3     @future(callout=true)
4     public static void sendAppointmentToEMR(Id appointmentId) {
5
6         Appointment__c appointment = [
7             SELECT Id,
8                 Name,
9                 Patient__c,
10                Doctor__c,
11                Appointment_Date__c,
12                Start_Time__c,
13                End_Time__c,
14                Status__c
15            FROM Appointment__c
16            WHERE Id = :appointmentId
17            LIMIT 1
18        ];
19
20        HttpRequest request = new HttpRequest();
21
22        request.setEndpoint('https://example.com/api/appointments');
23        request.setMethod('POST');
24        request.setHeader('Content-Type', 'application/json');
25
26        Map<String, Object> appointmentData = new Map<String, Object>{
27            'appointmentId' => appointment.Id,
28            'patientId' => appointment.Patient__c,
29            'doctorId' => appointment.Doctor__c,
30            'appointmentDate' => appointment.Appointment_Date__c,
31            'startTime' => appointment.Start_Time__c,
32            'endTime' => appointment.End_Time__c,
33            'status' => appointment.Status__c
34        };
35
36        request.setBody(JSON.serialize(appointmentData));
37
38        Http http = new Http();
39        HttpResponse response = http.send(request);
40
41        System.debug('EMR Response Status: ' + response.getStatusCode());
42        System.debug('EMR Response Body: ' + response.getBody());
43    }
44 }
```

What is a Lightning Web Component?

Salesforce's modern UI framework — used here to build the patient- and doctor-facing screens.

In Simple Terms

LWC (Lightning Web Component) is how Salesforce builds fast, reusable, modern screens inside its platform — similar to how apps like Gmail or banking apps have interactive buttons, forms, and calendars.

Instead of a plain, static page, an LWC screen reacts instantly: click a date, and available slots appear right away — no page reload, no waiting.

For this project, LWC is the layer patients, doctors, and admins actually see and click on every day.

Why LWC for This Project



Real-Time Feel

Slot booking and dashboards update instantly, without refreshing the page.



Built for Reuse

The same component (e.g. calendar) can be reused across scheduler, dashboard, and admin screens.



Native to Salesforce

Runs directly inside Salesforce, so it stays connected to Patient, Doctor, and Appointment data with no extra integration.



Fast & Lightweight

Built on modern web standards, so pages load quickly even with live data.

Lightning Web Components — In Detail

Three purpose-built components deliver the day-to-day user experience.

1

Appointment Scheduler

Used by: Patients & Admins

A drag-and-drop calendar for fast, visual booking.

- Drag & drop to pick a slot
- Blocks slot instantly on booking
- Visual, calendar-style layout

2

Doctor Availability Grid

Used by: Patients

Search and filter doctors by specialty and open slots.

- Filter by specialty
- Shows real-time availability
- Quick doctor-to-slot matching

3

Patient Dashboard

Used by: Patients

A single home screen for a patient's appointment activity.

- View full visit history
- Reschedule in a click
- Cancel upcoming appointments

LWC — Appointment Scheduler

Drag-and-drop calendar component used by patients and admins to book a slot.

1 Data Source

Wires to Apex to pull the doctor's open Time Slot records.

2 Interaction

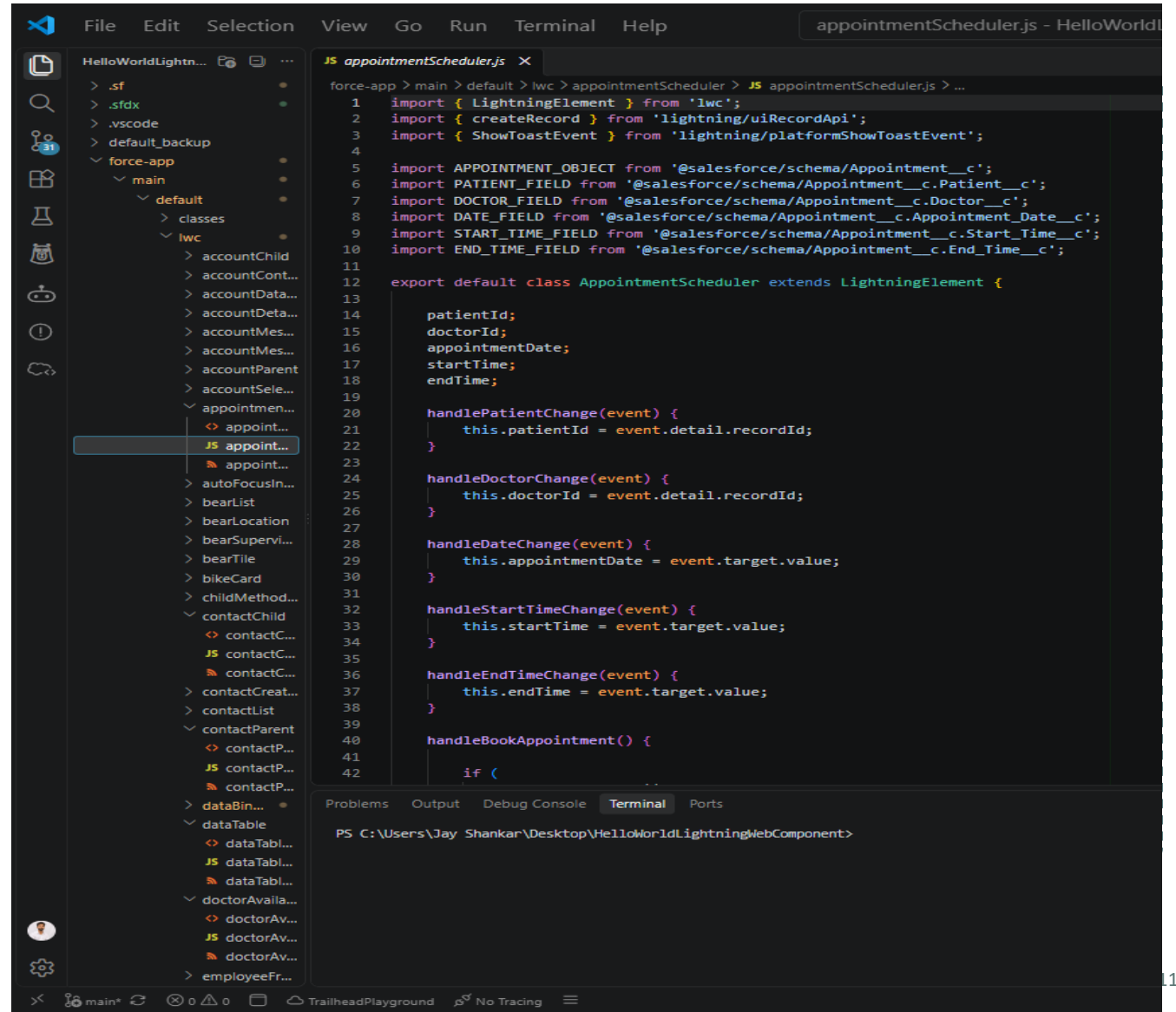
User drags an appointment onto an open slot on the calendar.

3 On Booking

Calls Apex to create the Appointment and hand off to the Flow/Trigger.

4 Feedback

Slot disappears from the grid instantly once booked.



LWC — Doctor Availability Grid

Search-and-filter component patients use to find the right doctor and slot.

1 Filter Input

A specialty picklist/search box drives the results shown.

2 Data Source

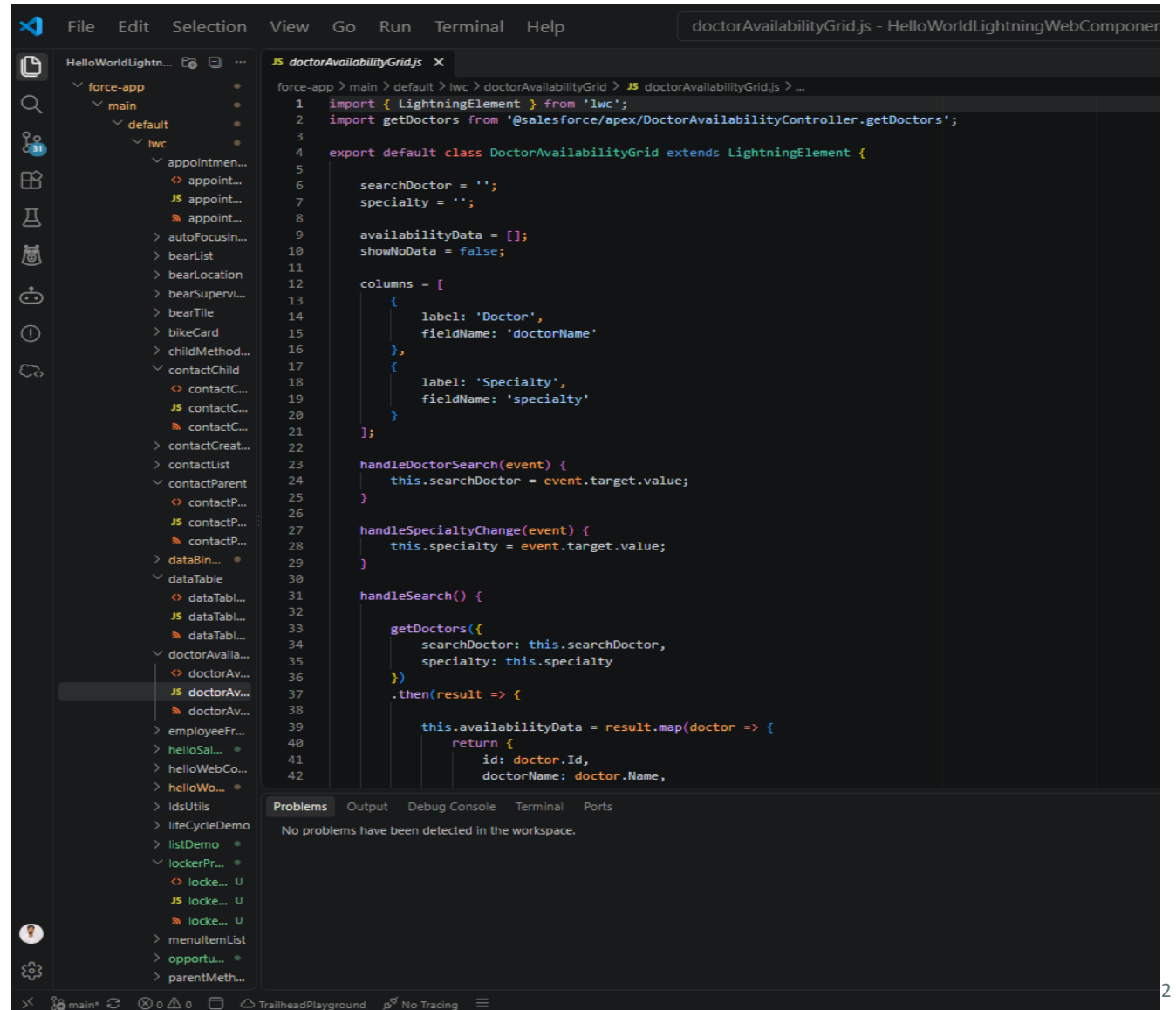
Calls Apex to fetch doctors matching the chosen specialty.

3 Live Availability

Each doctor row shows their next open Time Slot in real time.

4 Next Step

Selecting a doctor hands the patient into the Appointment Scheduler.



The screenshot shows the Visual Studio Code editor with a project named 'HelloWorldLightningWebComponent'. The file explorer on the left shows a directory structure: 'force-app' > 'main' > 'default' > 'lwc' > 'doctorAvailabilityGrid'. The main editor displays the 'doctorAvailabilityGrid.js' file, which is a Lightning Web Component (LWC) that extends 'LightningElement'. The code includes imports for 'LightningElement' and 'getDoctors' from '@salesforce/apex/DoctorAvailabilityController.getDoctors'. It defines a class 'DoctorAvailabilityGrid' with properties 'searchDoctor' and 'specialty', and a data array 'availabilityData'. The component has three methods: 'handleDoctorSearch', 'handleSpecialtyChange', and 'handleSearch'. The 'handleSearch' method calls 'getDoctors' with the current search and specialty values, and then maps the results to the 'availabilityData' array. The 'Problems' panel at the bottom shows no errors.

```
1 import { LightningElement } from 'lwc';
2 import getDoctors from '@salesforce/apex/DoctorAvailabilityController.getDoctors';
3
4 export default class DoctorAvailabilityGrid extends LightningElement {
5
6     searchDoctor = '';
7     specialty = '';
8
9     availabilityData = [];
10    showNoData = false;
11
12    columns = [
13        {
14            label: 'Doctor',
15            fieldName: 'doctorName'
16        },
17        {
18            label: 'Specialty',
19            fieldName: 'specialty'
20        }
21    ];
22
23    handleDoctorSearch(event) {
24        this.searchDoctor = event.target.value;
25    }
26
27    handleSpecialtyChange(event) {
28        this.specialty = event.target.value;
29    }
30
31    handleSearch() {
32
33        getDoctors({
34            searchDoctor: this.searchDoctor,
35            specialty: this.specialty
36        })
37        .then(result => {
38
39            this.availabilityData = result.map(doctor => {
40                return {
41                    id: doctor.Id,
42                    doctorName: doctor.Name,
```

LWC — Patient Dashboard

A patient's home screen for viewing, rescheduling, and cancelling visits.

1 Data Source

Wires to Apex to pull the logged-in patient's appointment history.

2 Reschedule

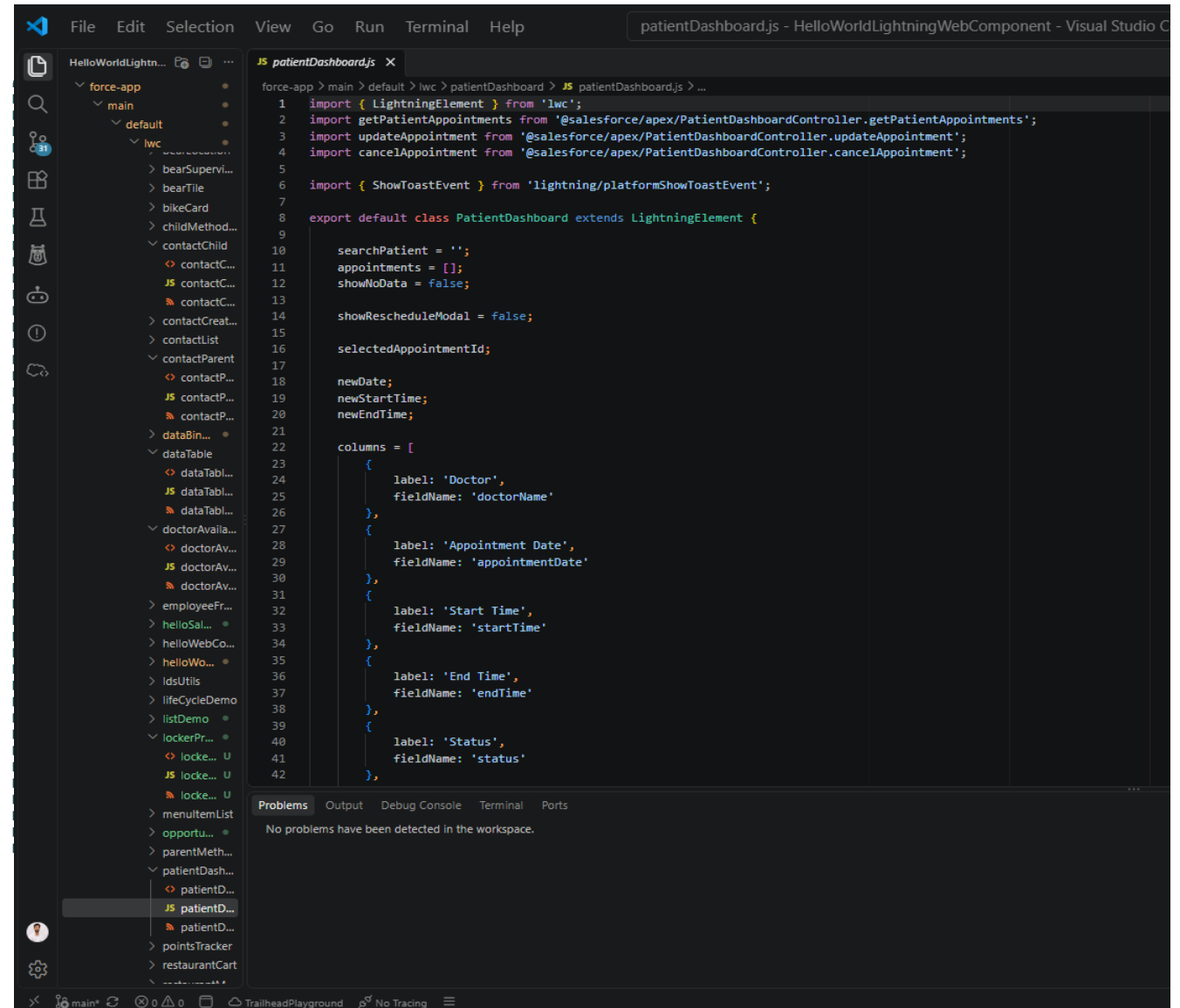
Opens the Scheduler pre-filled, then updates the existing record.

3 Cancel

Calls Apex to cancel and automatically frees up the Time Slot.

4 Visibility

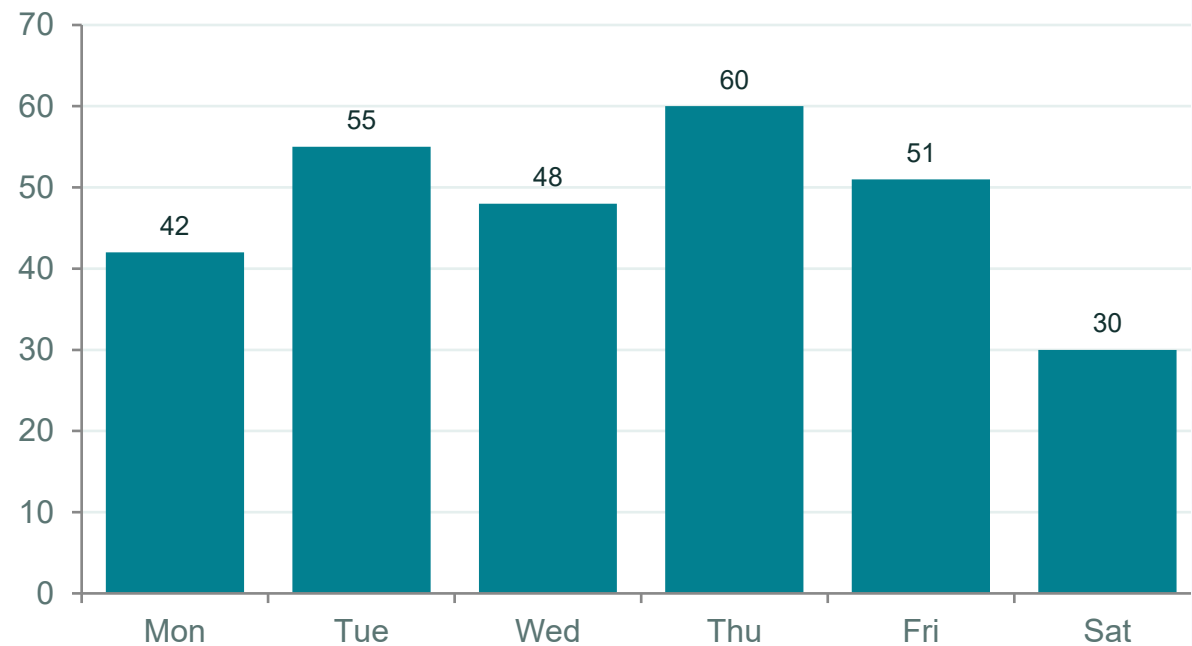
Past and upcoming appointments are separated for quick scanning.



Reports & Dashboards

Operational visibility into scheduling volume, outcomes, and doctor demand.

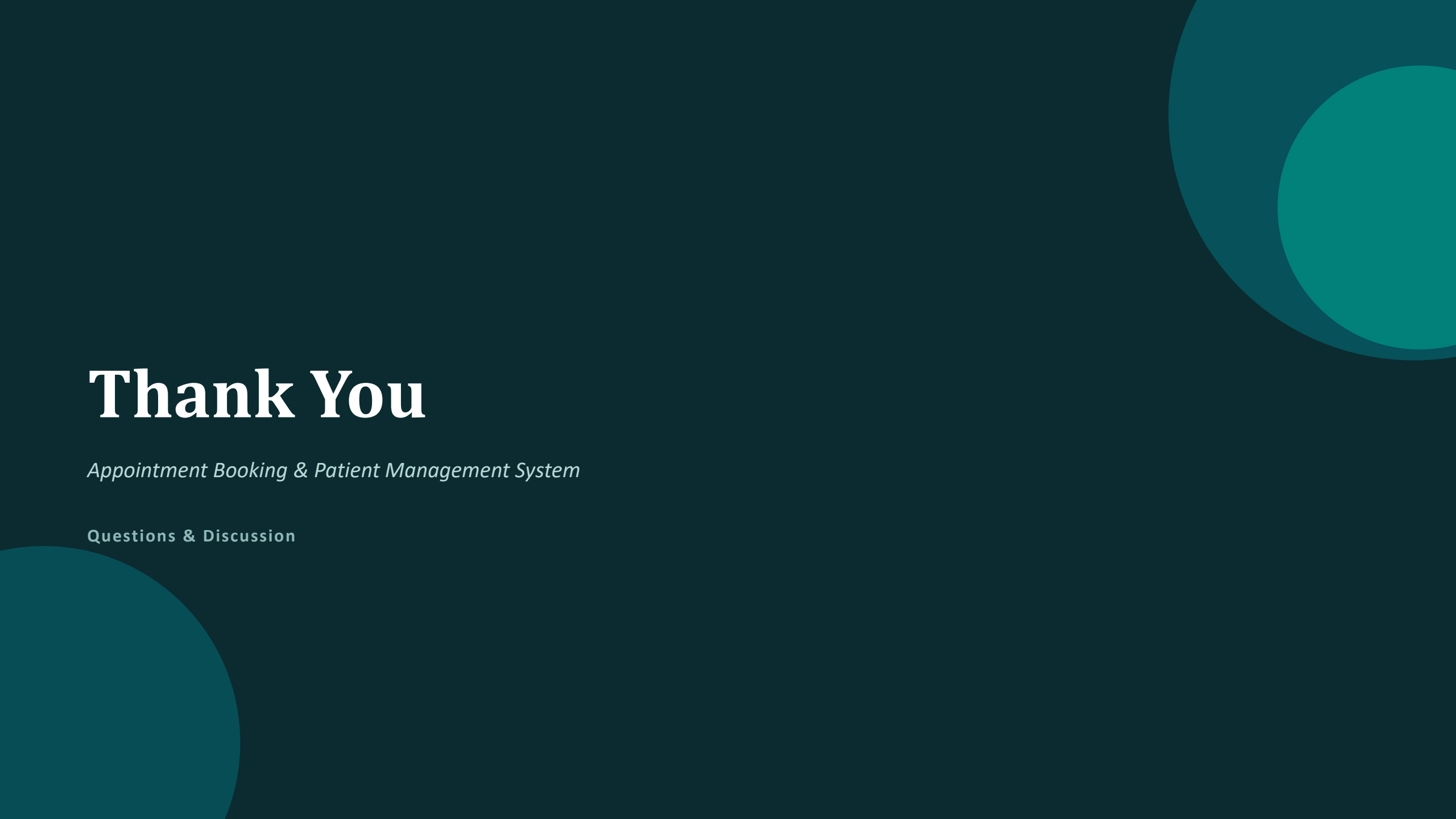
Appointments per Week (Illustrative)



Why This Matters

- One connected system replaces manual, fragmented scheduling.
- Fewer no-shows and double-bookings through automated safeguards.
- Real-time EMR sync keeps clinical records current.
- Clear dashboards give leadership ongoing operational visibility.

Ready for review and next-step planning.



Thank You

Appointment Booking & Patient Management System

Questions & Discussion